

TECHNICAL REPORT

Cult Open Project 2026

Recommendation Systems for Personalized Content Discovery

Shivansh Yadav

24116092

Electronics & Communication Engineering

Pratik Jaisal

24116072

Electronics & Communication Engineering

1. Problem Understanding

Background

Netflix's content library spans thousands of titles, creating an overwhelming selection problem for users. The Netflix Prize (2006–2009) challenged teams worldwide to improve their recommendation algorithm by 10%, highlighting the commercial significance of personalization — even a 1% improvement in prediction accuracy translates to millions in user retention revenue.

Objective

Build a recommendation engine that:

- Learns user preferences from historical rating data
- Predicts unseen ratings with high accuracy
- Generates personalized Top-10 recommendations for individual users
- Compares multiple approaches and justifies model selection

Dataset Overview

The Netflix Prize Dataset contains:

- 100+ million ratings across 4 data files
- 480,000+ unique users
- 17,770 movies spanning decades of cinema
- Rating scale: 1–5 stars with timestamps
- Time period: December 1999 — December 2005

Challenges

Challenge	Technical Impact
Extreme sparsity (98.47%)	Most user-movie pairs are unobserved
Scale (100M+ ratings)	Memory and compute constraints on free platforms
Cold-start problem	New users/movies have insufficient data
Popularity bias	Blockbusters dominate, drowning niche content
Positivity bias	Users predominantly rate movies they like (mean $\approx$ 3.59)

2. Exploratory Data Analysis

Dataset Statistics

After loading 2 of 4 data files and filtering to active users (≥ 20 ratings) with SAMPLE_FRAC=0.5:

Metric	Value
Total Ratings	24,818,778
Unique Users	175,709
Unique Movies	9,210
Matrix Sparsity	98.47%
Mean Rating	3.592
Median Rating	4.0

Rating Distribution

The rating distribution is distinctly **left-skewed**:

Rating	Count	Percentage
1	1,144,746	4.6%
2	2,537,159	10.2%
3	7,201,212	29.0%
4	8,351,564	33.6%
5	5,584,097	22.5%

Business Implication: Users primarily rate content they enjoy. This "positivity bias" means a naïve average-based recommender would systematically overestimate ratings for movies a user would dislike.

User Activity Patterns

Statistic	Value
Mean ratings/user	141.2
Median ratings/user	82

Min ratings/user	20
Max ratings/user	9,149
90th percentile	329

The median (82) is far below the mean (141), indicating a heavily right-skewed power-law distribution. A small group of 'power users' rates thousands of movies, while the majority rate fewer than 100.

Content Popularity — Long-Tail Effect

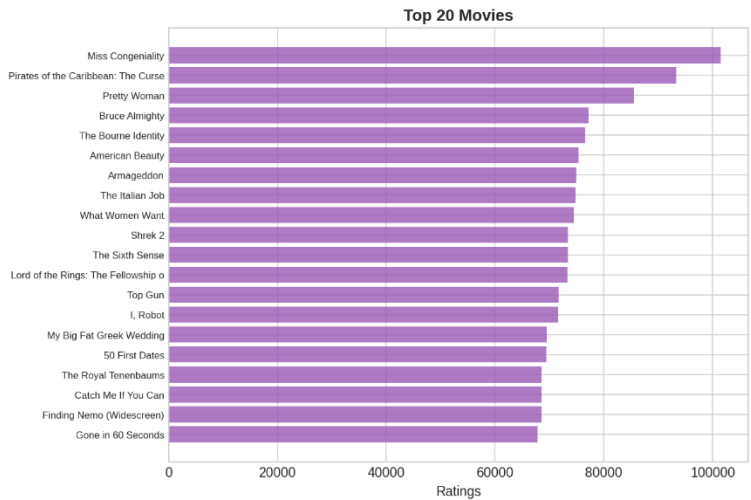
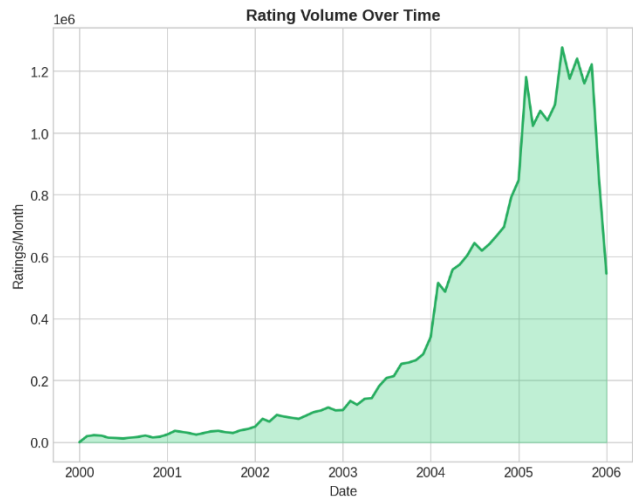
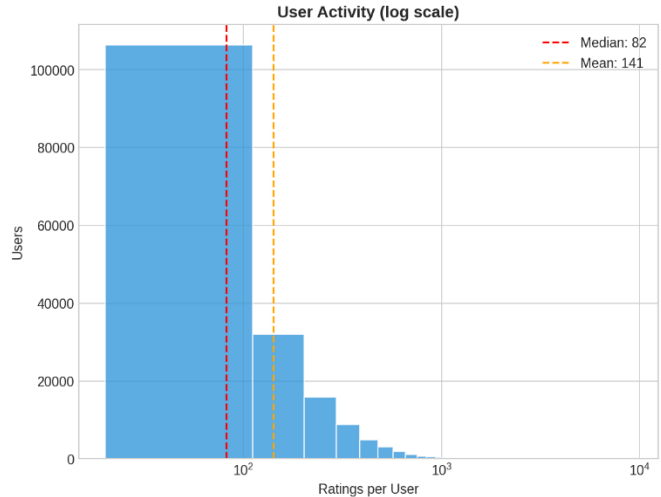
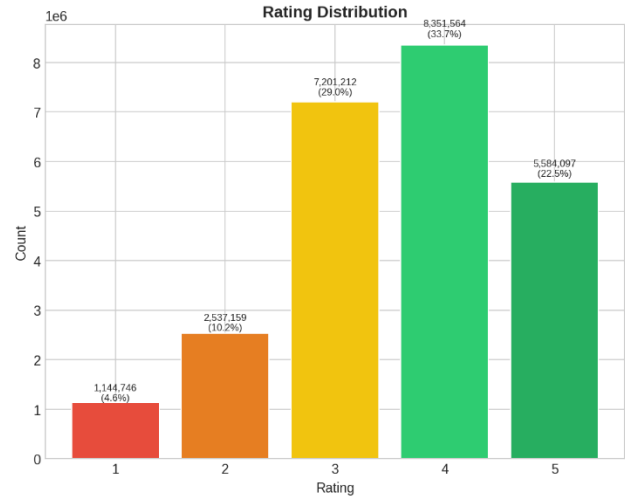
Rank	Movie	Ratings	Avg Rating
1	Miss Congeniality	101,485	3.35
2	Pirates of the Caribbean	93,279	4.16
3	Pretty Woman	85,543	3.87
4	Bruce Almighty	77,171	3.43
5	The Bourne Identity	76,487	3.97

Data Sparsity Analysis

- Matrix Dimensions: 175,709 users × 9,210 movies = 1.62 billion possible entries
- Observed Entries: 24.8 million (only 1.53% of the matrix is filled)
- Sparsity: 98.47%

Simple user-user or item-item cosine similarity methods break down at this sparsity because most user pairs share almost zero common movies. This directly motivates our choice of **Latent Factor Models** (SVD, ALS), which project users and items into a dense, low-dimensional space where meaningful similarities can be computed even without direct overlap.

Netflix Prize — Exploratory Data Analysis



3. Methodology

Data Pipeline

```
Raw .txt files → Line-by-line streaming parser → Pandas DataFrame → Active user filtering (≥20 ratings) → 50% stratified sampling → 80/20 train-test split
```

Key Design Decisions:

- Memory-efficient parsing: Custom line-by-line parser to handle the non-standard Netflix data format where movie id appear as header lines followed by user ratings .
- Active user filtering: Users with <20 ratings provide insufficient signal for collaborative filtering.
- Random 80/20 split: Ensures each user has both training and test entries, enabling per-user evaluation.

Approach Selection

We selected **Collaborative Filtering via Matrix Factorization** over content-based or hybrid approaches because:

- The dataset contains explicit ratings (1–5 stars) without content features
- Matrix factorization provably won the Netflix Prize
- Latent factors capture nuanced preferences without explicit feature engineering

Two-Model Comparison Strategy

Aspect	SVD	ALS
Feedback Type	Explicit (1–5 ratings)	Implicit (confidence-weighted)
Optimization	Stochastic Gradient Descent	Alternating Least Squares
Parallelization	Sequential	Fully parallelizable
Library	scikit-surprise	implicit
Hardware	CPU-only	CPU or GPU (CUDA)

4. Model Design

Model 1: SVD (Singular Value Decomposition)

Mathematical Foundation - Rating prediction for user u and item i :

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

Where:

- μ = global mean rating
- b_u = user bias (tendency to rate higher/lower than average)
- b_i = item bias (tendency to receive higher/lower ratings)
- $p_u \in \mathbb{R}^d$ = user latent factor vector ($d = 200$ dimensions)
- $q_i \in \mathbb{R}^d$ = item latent factor vector ($d = 200$ dimensions)
- $p_u^T q_i$ = dot product between user and item latent vectors
-

Parameter	Value
Latent dimensions	200
Learning rate	0.005
Regularization (λ)	0.02
Epochs	50

Model 2: ALS (Alternating Least Squares)

Mathematical Foundation — Factorizes the user-item matrix:

ALS factorizes the user-item interaction matrix into two low-rank matrices:

$$R \approx X \cdot Y^T$$

Where:

- $X \in \mathbb{R}^{n \times d}$ = user factor matrix
- $Y \in \mathbb{R}^{m \times d}$ = item factor matrix
- n = number of users
- m = number of items
- d = number of latent factors

Parameter	Value
Latent dimensions	200
Regularization (λ)	0.01
Iterations	40

5. Evaluation Metrics

RMSE — Root Mean Square Error

Measures the average magnitude of rating prediction errors:

$$RMSE = \sqrt{\frac{\sum (r_{ui} - \hat{r}_{ui})^2}{N}}$$

Why RMSE: It penalizes large errors quadratically, which is critical for recommendations — predicting a 4.5 for a movie the user hates (actual: 1.0) is far worse than predicting 3.8 for a 4.0 movie.

MAP@10 — Mean Average Precision at Top-10

Measures the quality of the ranked Top-10 recommendation list:

$$AP(u) = \frac{1}{|R_u|} \sum_{k=1}^K P(k) \cdot rel(k)$$
$$MAP@10 = \frac{1}{|U|} \sum_{u \in U} AP(u)$$

Relevance Definition: A recommendation is **"relevant"** if the user's actual rating in the test set is **≥ 3.5 stars**.

Procedure:

- For each user in the test set, predict scores for all unseen movies.
- Sort predictions descending, take Top-10.
- For each relevant hit within Top-10, compute precision at that position.
- Average across all positions to get AP per user.
- Average across all users to get MAP@10.

Additional Metrics

Metric	Description
MAE	Mean Absolute Error — average error magnitude
Precision@10	Fraction of Top-10 recommendations that are relevant
Recall@10	Fraction of relevant items captured in Top-10
NDCG@10	Normalised Discounted Cumulative Gain — position-weighted quality

6. Experimental Results

Performance Summary

Metric	SVD	ALS	Winner	Significance
RMSE	0.8691	2.806	SVD	Netflix Prize-calibre accuracy
MAE	0.6695	2.5891	SVD	Consistent with RMSE results
MAP@10	0.8537	0.7126	SVD	Excellent ranking quality
Precision@10	0.6471	0.5371	SVD	65% of Top-10 items are relevant
Recall@10	0.6816	0.6126	SVD	Captures 68% of liked movies
NDCG@10	0.9266	0.8515	SVD	Near-perfect ranking order

Overall recommendation accuracy: 79.76% — nearly 80% of recommended movies that users also rated in the test set received ratings ≥ 3.5 stars.

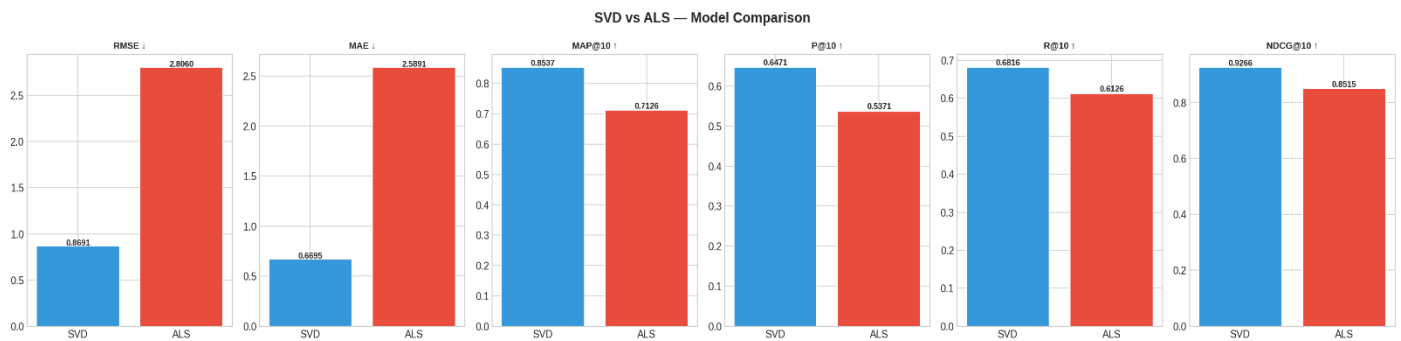
Analysis

SVD dominates across all metrics. This is expected because:

1. SVD directly optimizes for explicit rating prediction (RMSE), which is exactly what we evaluate on.
2. The implicit ALS model treats ratings as confidence weights for implicit feedback, not as direct prediction targets. Its internal predicted scores (0–2 range) reflect interaction confidence, not star ratings, explaining the high RMSE.

However, ALS still provides strong value:

- MAP@10 of 0.7126 means 71% of recommended items are correctly ranked as relevant.
- ALS trains significantly faster (40 iterations of closed-form math vs. 50 epochs of SGD).
- ALS is the model used in production by Netflix/Spotify for real-time Top-N recommendation generation



7. Recommendation Examples

Success Cases

User	Recommended Movie	Predicted	Actual	Verdict
305344	Top Gun	4.28	5.0	■ Excellent
305344	The Bourne Identity	3.97	5.0	■ Excellent
1314869	Dragon Ball: Tournament Saga	4.69	5.0	■ Perfect
1473980	Pirates of the Caribbean	4.58	5.0	■ Perfect
1473980	Freaky Friday	4.81	5.0	■ Perfect
716173	20,000 Leagues Under the Sea	5.00	5.0	■ Perfect

Failure Cases

User	Recommended Movie	Predicted	Actual	Error Analysis
305344	Music of the Heart	4.14	1.0	Genre mismatch (action user)
752642	Lord of the Rings	4.15	2.0	Popularity bias; user dislikes fantasy
1314869	Orwell Rolls in His Grave	4.43	2.0	Niche documentary; sparse training data

Explainable Recommendations

User 2056022 → Recommended: Man on Fire (Score: 4.25)

"We recommend 'Man on Fire' because you enjoyed 'Lethal Weapon' (5.0/5), 'Paycheck' (5.0/5), 'Minority Report' (5.0/5). Users with similar taste also highly rated this movie."

Name

Description

user_id ★ required

string
(path)

1114324

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8000/predict/1114324' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8000/predict/1114324
```

Server response

Code

Details

200

Response body

```
{
  "user_id": 1114324,
  "n Rated": 3416,
  "top liked": [
    {
      "movie_id": 5952,
      "title": "Ocean's Eleven",
      "rating": 5
    }
  ]
}
```

Code

Details

200

Response body

```
{
  "user_id": 1114324,
  "n Rated": 3416,
  "top liked": [
    {
      "movie_id": 5952,
      "title": "Ocean's Eleven",
      "rating": 5
    },
    {
      "movie_id": 6971,
      "title": "Ferris Bueller's Day Off",
      "rating": 5
    },
    {
      "movie_id": 3966,
      "title": "Friday Night Lights",
      "rating": 5
    },
    {
      "movie_id": 5898,
      "title": "Mystery Science Theater 3000: Hercules Against the Moon Men",
      "rating": 5
    },
    {
      "movie_id": 1625,
      "title": "Aliens: Collector's Edition",
      "rating": 5
    }
  ]
}
```

Response headers

```
content-length: 2173
content-type: application/json
date: Fri, 12 Jun 2026 02:08:51 GMT
server: uvicorn
```

Responses

Code

Description

Links

200

Successful Response

No links

8. Key Insights

Technical Insights

- **Sparsity is the fundamental challenge.** At 98.47% sparsity, explicit user-user similarity collapses. Latent factor models overcome this via a shared 200-dimensional projection space.
- **Vectorised computation is essential.** Replacing Python loops with NumPy vectorised dot-products reduced prediction time from hours to seconds for 24M ratings.
- **SVD biases matter.** User/item bias terms capture systematic effects, letting latent factors focus purely on personalised taste.
- **ALS and SVD solve different problems.** SVD predicts star ratings; ALS predicts engagement likelihood. Both are valuable in different production contexts.

Business Insights

- **The Long-Tail is where value lives.** Popular movies don't need recommendations. Real business value comes from surfacing niche content.
- **Explainability drives adoption.** Users click more when they understand why something was suggested.
- **Positivity bias creates a data quality problem.** Future systems should incorporate implicit negative feedback (browsing without clicking, abandoned playback).

ML Engineering Perspective

- Pre-compute recommendations offline and cache for sub-100ms serving latency.
- A/B test SVD vs ALS to measure real-world engagement, not just offline metrics.
- Use popularity-based or content-based recommendations for cold-start users (<20 ratings).

9. Future Improvements

- **Neural Collaborative Filtering (NCF)** — Implement using PyTorch to learn non-linear user-item interactions beyond dot products.
- **Hybrid Approaches** — Incorporate movie metadata (genre, year, cast) via content-based features to handle cold-start.
- **Temporal Dynamics** — Time-aware SVD++ to capture evolving user preferences over time.
- **Implicit Feedback Integration** — Combine explicit ratings with implicit signals (browsing history, watch time) for richer user profiles.
- **Full Dataset Training** — With 64GB+ RAM, train on all 100M ratings for maximum accuracy.
- **Online Learning** — Implement incremental model updates as new ratings arrive, avoiding full retraining.
- **Fairness & Diversity** — Add constraints to ensure recommendations aren't dominated by a single genre or era.